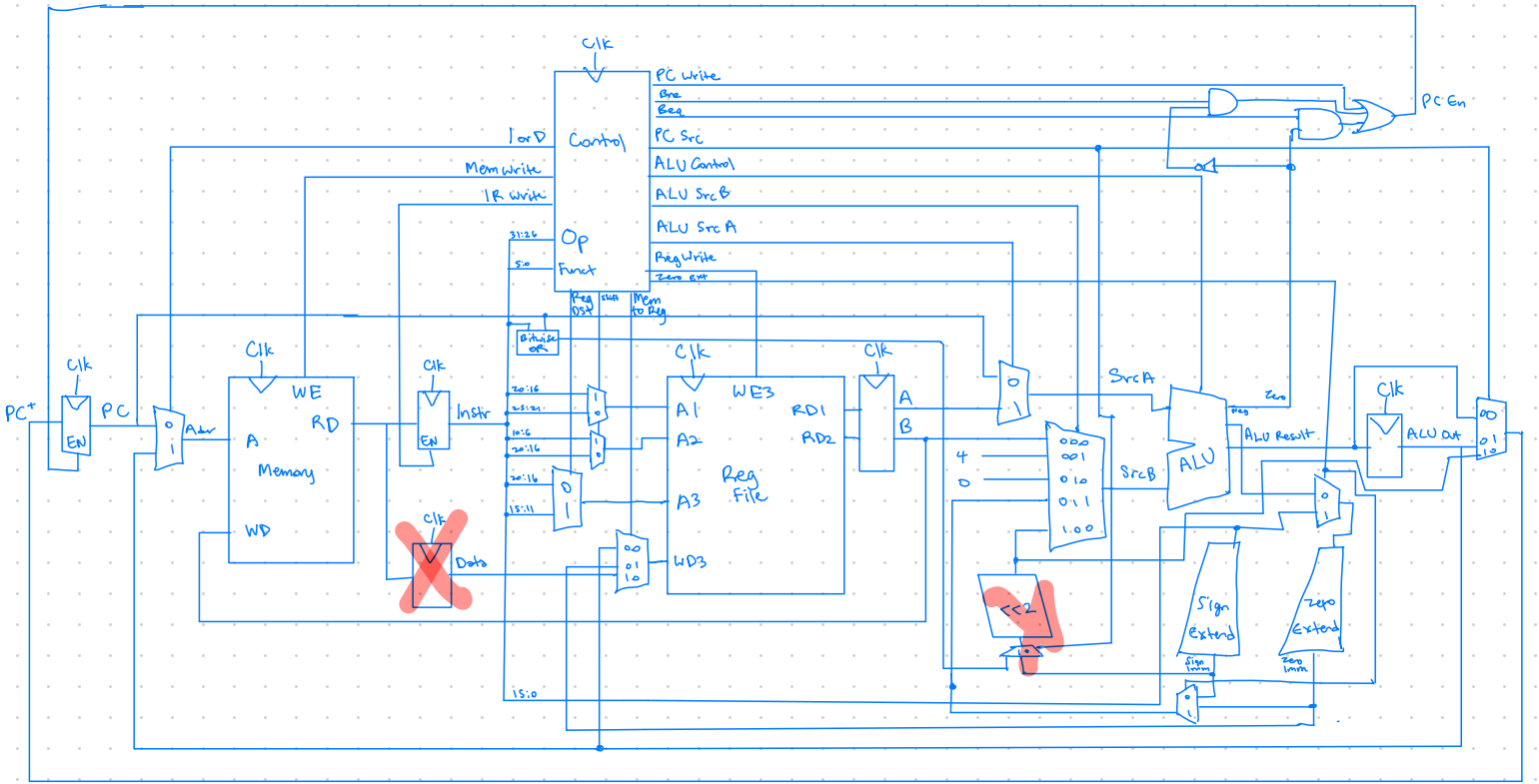
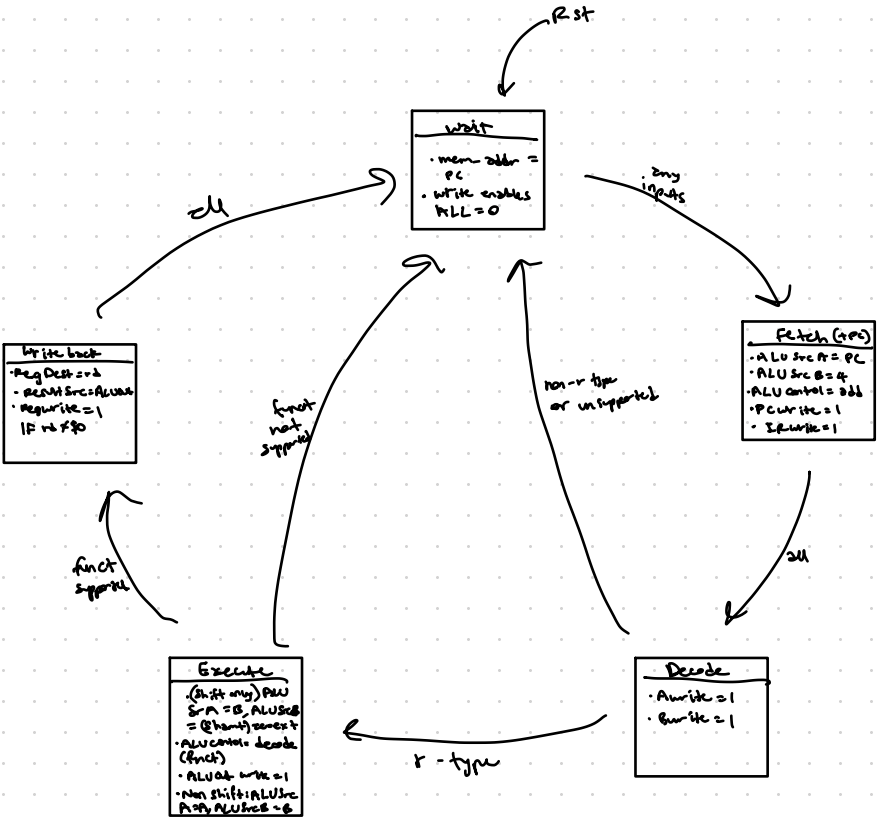



- and/or/xor/nor: Assume ALU can do bitwise logic
- jumps: bitwise or/and blocks
- srl/sra/sll: Added diff. bit line access options to src A1 & A2 reads since no shamt, etc.
- jr: Added 0 option to Src B in case we want same address
- bne: Add gate logic to control unit so $\overline{\text{Zero}}$ output will be ok.
- slt: use subtraction's negative flag
- andi/ori/xori: allow select 0-extend for imm


Rst
Wait
· mem addr = PC
· write enables ALL = 0
· any inputs
Fetch (+PC)
· ALU src A = PC
· ALU src B = 4
· ALU control = add
· PC write = 1
· I&R write = 1
all
Decode
· A write = 1
· B write = 1
R - type
Execute
· (Shift only) ALU Src A = B, ALUSrcB = (Shamt) zero-ext
· ALU control = decode (funct)
· ALUOut write = 1
· Non shift: ALUSrc A = A, ALUSrcB = B
funct supported
Write back
· RegDest = rd
· RegWrSrc = ALUout
· RegWrite = 1 IF rd ≠ $0
all
funct not supported
non-r type or unsupported